High-Performance Pipelined RV32I Processor Core — Design Report

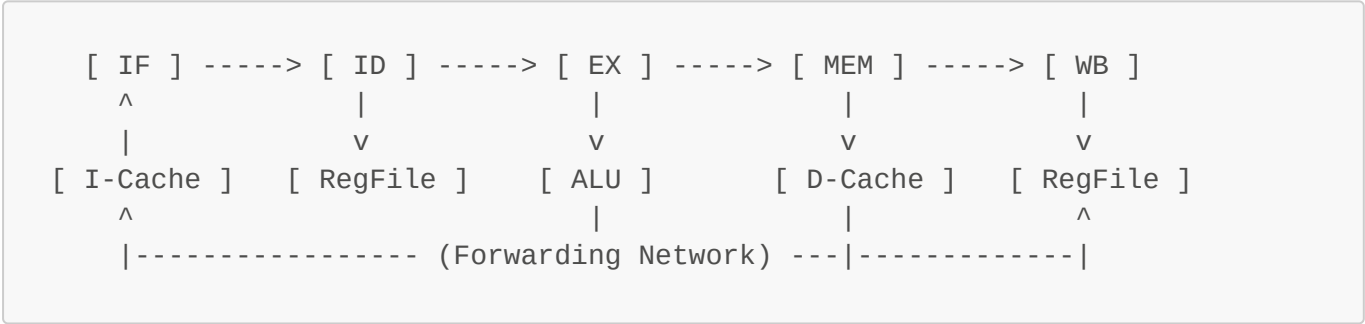
1. Project Overview

This document presents the detailed architectural design, implementation, and verification results of a high-performance 5-stage pipelined RV32I RISC-V processor core. The design features dynamic branch prediction, split Level-1 (L1) instruction and data caches, and an integrated SPI peripheral master.

The processor is synthesized and targeted for the Xilinx Kintex-7 `xc7k325tfbv900-3` FPGA (Speed Grade -3) utilizing Vivado v.2026.1 as the primary EDA toolchain.

2. Architecture

The core implements a classic 5-stage RISC pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory (MEM), and Writeback (WB) stages.



2.1 Pipeline Stages

2.1.1 Fetch (IF) — `fetch.v`

The Fetch stage handles Program Counter (PC) generation and instruction retrieval. A priority multiplexer selects the next PC, prioritizing mispredict recovery, followed by stall holds, predicted taken branches, and finally the sequential PC+4.

The stage features a **256-entry dynamic branch predictor**:

- **Branch History Table (BHT)**: Utilizes 2-bit saturating counters (Strongly Not Taken → Weakly Not Taken → Weakly Taken → Strongly Taken).
- **Branch Target Buffer (BTB)**: Caches 32-bit branch target addresses.
- **Tag Table**: Provides 22-bit aliasing protection tags.
- **Valid Table**: 1-bit entry validity (cleared upon reset).

The predictor is indexed by `pc_reg[9:2]` and tagged by `pc_reg[31:10]`. A branch is predicted taken if `valid && tag_match && bht[1]` is true. Updates occur synchronously via `actual_branch_valid` from the Decode stage; a tag mismatch resets the counter to WT/WNT. All prediction tables utilize distributed RAM (`(* ram_style = "distributed" *)`). The canonical NOP used is `32'h00000013` (addi x0, x0, 0).

2.1.2 Decode (ID) — `decode.v`

The Decode stage unpacks the 32-bit instruction into opcode[6:0], funct3[2:0], funct7[6:0], rs1[19:15], rs2[24:20], and rd[11:7]. Immediates are generated for U, I, S, B, and J type instructions. For the `LUI` instruction, the source register `rs1_addr` is forced to `5'b00000` (x0).

The control unit decodes:

- **ALU Operations**: 0000=ADD, 0001=SUB, 0010=AND, 0011=OR, 0100=XOR, 0101=SLT
- **Writeback Selection**: 00=ALU, 01=Memory, 10=PC+4

Crucially, **early branch resolution** is performed here utilizing forwarded operands (`rs1_data_fwd`, `rs2_data_fwd`):

- BEQ: rs1 == rs2
- BNE: rs1 != rs2
- BLT: $signed(rs1) < $signed(rs2)
- BGE: $signed(rs1) >= $signed(rs2)
- JAL: Unconditional jump

The actual target is calculated as `if_id_pc + imm_val`. A mispredict is flagged when `actual_branch_valid && (pred_taken != actual_branch_taken) && !stall`.

2.1.3 Execute (EX) — `execute.v`

The Execute stage contains the Arithmetic Logic Unit (ALU) which supports ADD, SUB, AND, OR, XOR, and signed SLT operations. 3-to-1 forwarding multiplexers are present on both ALU inputs, selecting data from the RegFile, MEM stage, or WB stage. The second ALU operand (`alu_in2`) selects between the forwarded `rs2` value and the immediate value based on the `id_alu_src` control signal.

2.1.4 Memory (MEM) — `memory.v`

The Memory stage routes requests to either the L1 Data Cache or the Memory-Mapped I/O (MMIO) controller.

- Address demultiplexing uses the MSB `ex_mem_alu_result[31]`: MMIO accesses (≥0x80000000) vs Cache accesses (<0x80000000).
- The read data multiplexer selects `is_mmio ? mmio_rdata : cache_rdata`.
- Pipeline stalls are generated if `(is_cache && !cache_hit) || (is_mmio && !mmio_ready)`.

2.1.5 Writeback (WB) — `writeback.v`

A purely combinational multiplexer routes the final result to the Register File write port and the pipeline forwarding bus, selecting from the ALU result (00), Memory data (01), or PC+4 (10).

2.2 Hazard Unit — `hazard.v`

The hazard unit ensures pipeline correctness via data forwarding and stalling mechanisms.

- **EX Forwarding** (`fwd_a`, `fwd_b`): MEM priority (2'b10) > WB (2'b01) > RegFile (2'b00).

- **ID Forwarding** (`rs1_data_fwd`, `rs2_data_fwd`): Same priority scheme, feeding the early branch comparators.
- **Load-Use Stall**: Triggered when `ex_mem_read && rd_match`.
- **Branch Data Stall**: Triggered if a branch in ID depends on an instruction currently in EX (requiring ALU calculation) or a Load in MEM.

Pipeline controls are defined as follows:

- `stall_if = stall_mem || stall_icache || id_hazard_stall`
- `stall_id = stall_mem || stall_icache`
- `stall_ex = stall_mem`
- `flush_ex = (id_hazard_stall || stall_icache) && !stall_mem`

2.3 Register File — `regfile.v`

The processor utilizes a 32×32-bit register file synthesized as distributed RAM (`(* ram_style = "distributed" *)`). It provides two combinational read ports and one synchronous write port. An internal Read-After-Write (RAW) bypass mechanism ensures that if `rs == rd && reg_write`, the `rd_data` is forwarded directly. Register `x0` is hardwired to zero (reads return 0, writes are suppressed).

2.4 L1 Caches

2.4.1 Instruction Cache — `icache.v`

The L1 I-Cache is a 64-entry (2^6) direct-mapped, read-only cache with a 1-word (32-bit) line size.

- Address Breakdown: index = addr[7:2], tag = addr[31:8].
- Hit Condition: `req && valid[index] && (tag_array[index] == tag)`.
- FSM States: IDLE → FETCH (assert mem_req on miss) → READY (1-cycle hit hold) → IDLE.
- Both data and tag arrays use distributed RAM.

2.4.2 Data Cache — `dcache.v`

The L1 D-Cache shares the same structural FSM and sizing parameters as the I-Cache (64-entry direct-mapped, 1-word line size).

- **Write Policy**: Write-Through + Write-Allocate.
- A write hit updates the cache and writes through to main memory.
- A write miss writes directly to BRAM and allocates the cache line with the write data.
- A read miss fetches data from BRAM and allocates the cache line.

2.5 Physical RAM — `ram.v`

Main memory is implemented using Dual-Port Harvard Block RAM (`(* ram_style = "block" *)`).

- Port A (I-Cache Interface): Synchronous read, 1-cycle latency.
- Port B (D-Cache Interface): Synchronous read/write, 1-cycle latency.
- Capacity: 1024 words per port (4 KB Instruction RAM + 4 KB Data RAM).
- Pre-initialized via `$readmemh(imem.hex, iram)`.

2.6 SPI Master Integration — `system.v`

A Memory-Mapped SPI Master is integrated at addresses ≥ 0x80000000. MMIO is single-cycle ready (`mmio_ready = 1'b1`).

Address	Register	Description
`0x8000_0000`	SPI Data	TX Data (write[7:0]) / RX Data (read[7:0])
`0x8000_0004`	SPI CTRL	Control/Status Register

SPI Control Register Bit Mapping:

-
-
- [2]: TX FIFO Full (Read-Only)
- [3]: TX FIFO Empty (Read-Only)
-
- [5]: RX Valid (Read-Only)
-

3. Supported Instruction Set

The processor supports a subset of the RV32I Base Integer Instruction Set, totaling 19 instructions.

Type	Instruction	Opcode	Funct3	Funct7 / Imm	Operation
R	ADD	0110011	000	0000000	rd ← rs1 + rs2
R	SUB	0110011	000	0100000	rd ← rs1 - rs2
R	AND	0110011	111	0000000	rd ← rs1 & rs2
R	OR	0110011	110	0000000	rd ← rs1 | rs2
R	XOR	0110011	100	0000000	rd ← rs1 ^ rs2
R	SLT	0110011	010	0000000	rd ← (rs1 <s rs2) ? 1 : 0
I	ADDI	0010011	000	Immediate	rd ← rs1 + imm
I	ANDI	0010011	111	Immediate	rd ← rs1 & imm
I	ORI	0010011	110	Immediate	rd ← rs1 | imm
I	XORI	0010011	100	Immediate	rd ← rs1 ^ imm
I	SLTI	0010011	010	Immediate	rd ← (rs1 <s imm) ? 1 : 0
Load	LW	0000011	010	Immediate	rd ← MEM[rs1 + imm]
Store	SW	0100011	010	Immediate	MEM[rs1 + imm] ← rs2
Branch	BEQ	1100011	000	Immediate	if (rs1 == rs2) PC ← PC + imm
Branch	BNE	1100011	001	Immediate	if (rs1 ≠ rs2) PC ← PC + imm

Type	Instruction	Opcode	Funct3	Funct7 / Imm	Operation
Branch	BLT	1100011	100	Immediate	if (rs1 <s rs2) PC ← PC + imm
Branch	BGE	1100011	101	Immediate	if (rs1 ≥s rs2) PC ← PC + imm
Jump	JAL	1101111	N/A	Immediate	rd ← PC + 4; PC ← PC + imm
U	LUI	0110111	N/A	Immediate	rd ← imm << 12

4. FPGA Implementation Results

4.1 Timing Analysis

Target Clock: 125 MHz (8.000 ns period)

- **Worst Negative Slack (WNS)**: +1.685 ns (MET)
- **Worst Hold Slack (WHS)**: +0.075 ns (MET)
- **Total Negative Slack (TNS)**: 0.000 ns (0 failing endpoints out of 6,204)
- **Worst Pulse Width Slack (WPWS)**: +3.326 ns (MET)
- **Maximum Frequency (Fmax)**: ≈ 158 MHz (Tmin = 8.000 - 1.685 = 6.315 ns)

Critical Paths:

- *Setup Path*: `u_fetch/if_id_inst_reg[4]/C` → `u_fetch/bht_table` (11 logic levels, 5.782 ns data path delay: 0.975 ns logic / 4.807 ns routing)
- *Hold Path*: `u_execute/ex_mem_rs2_data_reg[11]/C` → `u_ram/dram_reg/DIADI[11]` (0 logic levels, 0.294 ns delay)

4.2 Resource Utilization

Resource	Utilized	Available	% Utilization
Slice LUTs	2,072	203,800	1.02%
- Logic	1,548	203,800	0.76%
- Distributed RAM	524	203,800	0.82%
Slice Registers (FFs)	987	407,600	0.24%
- FDCE (Async Reset)	914	-	-
- FDRE (Sync Reset)	66	-	-
- FDPE (Async Set)	7	-	-
Block RAM (RAMB36E1)	2	445	0.45%
DSP48E1	0	840	0.00%
Bonded IOBs	6	500	1.20%
Slices	634	50,950	1.24%

Other metrics: F7 Muxes: 120, F8 Muxes: 4, CARRY4: 67, BUFG: 1. IOBs used for `clk`, `rst_n`, `spi_miso`, `spi_cs_n`, `spi_mosi`, `spi_sclk`.

4.3 Power Analysis

- **Total On-Chip Power**: 202 mW
- **Static Power**: 158 mW (78.22%)
- **Dynamic Power**: 44 mW (21.78%)
  - Routing Signals: 17 mW (38.64% of dynamic)
  - Slice Logic: 13 mW (29.55% of dynamic)
  - Clocks: 11 mW (25.00% of dynamic)
  - Block RAM: 2 mW
  - I/O: 1 mW
- **Thermal Metrics**: Junction Temperature: 25.4°C / Thermal Margin: 74.6°C

Per-Module Dynamic Power Breakdown:

- `u_fetch` (Branch Predictor): 16 mW (36.36%)
- `u_execute` (ALU): 8 mW (18.18%)
- `u_decode`: 6 mW (13.64%)
- `spi_ctrl`, `u_dcache`, `u_icache`, `u_memory`, `u_ram`, `u_regfile`: 2 mW each (4.55% each)

4.4 Methodology Warnings

- **SYNTH-5 (48 violations)**: Distributed RAM mapped due to timing constraints.
- **SYNTH-6 (2 violations)**: Block RAM timing sub-optimal due to missing output pipeline register.
- **TIMING-18 (5 violations)**: Missing I/O delay constraints on `rst_n`, `spi_miso`, `spi_cs_n`, `spi_mosi`, and `spi_sclk`.

5. Simulation & Verification Results

Verification was performed using a self-checking testbench (`test_risc.v`) operating at 100 MHz (10 ns period) with a 20 ns active-low reset pulse. The simulation incorporates real-time Writeback transaction logging (`results.txt`), completion detection logic (waiting for `imem_addr ≥ 356` to settle for 5 cycles), and VCD waveform dumping.

The benchmark test program (`imem.s`) comprehensively exercises the pipeline across 6 phases:

1. ALU & RAW Hazard Interlocks
2. L1 Data Cache Store/Load, Miss/Hit, Load-Use Hazards
3. Fibonacci Sequence Loop
4. Euclidean GCD Algorithm
5. Modular Multiplication Loop
6. Bitwise Signatures & Cumulative Checksum

Verification Metrics

- **Register Checks**: ALL 31 REGISTER CHECKS PASSED (0 errors). Cumulative checksum (x31) matches expected `0xAEEABACA`.
- **Total Cycles**: 536

- **Instructions Retired**: 180
- **IPC (Instructions Per Cycle)**: 0.335
- **Branch Statistics**:
  - Branches Resolved: 85
  - Branch Mispredicts: 11
  - **Predictor Accuracy**: 87%

6. Future Work & Performance Optimization

Based on the architectural analysis and timing reports, the following non-invasive optimizations are recommended to improve IPC and maximum frequency:

1. **Increase Cache Line Size (4 or 8 Words)**: The current I/D caches utilize a 1-word (32-bit) line size, requiring a separate BRAM access for every new address. By widening the cache lines to 4-word (128-bit) lines with burst fills, I-Cache misses for sequential instruction streams would be drastically reduced. As the backend BRAM already supports single-cycle reads, this requires widening the internal data path or implementing sequential burst fill logic.

2. **Add I-Cache Prefetch / Next-Line Prefetch**: Because instruction streams exhibit high spatial locality, introducing a simple "prefetch next cache line" mechanism upon a current line hit would effectively hide almost all I-Cache miss latency for straight-line execution.

3. **Optimize Forwarding MUX Depth in Hazard Unit**: The current `rs1_data_fwd` and `rs2_data_fwd` paths in `hazard.v` exist as 3-level priority multiplexers that directly feed into the 32-bit branch comparators within `decode.v`. Flattening these structures into a single-level MUX, utilizing one-hot encoding for forwarding selects, would significantly reduce combinational depth on this timing path.

Pipeline Depth Trade-off (Branch Resolution)

Currently, branches are resolved early in the ID stage. Relocating branch resolution to the EX stage introduces an architectural trade-off:

- **Pros**: It would break the 11-level critical setup path originating in IF/ID, increasing the achievable Fmax. (This approach is employed in production cores such as the ARM Cortex-M4).
- **Cons**: It increases the branch misprediction penalty from 1 cycle to 2 cycles. Furthermore, it is a highly invasive change requiring modifications across `decode.v`, `hazard.v`, `fetch.v`, and `system.v`.
- **Conclusion**: Given that the current design meets timing comfortably with a +1.685 ns setup slack at 125 MHz, this structural change is only recommended if higher frequency targets (e.g., > 160 MHz) are mandated in future revisions.